UNIVERSITÉ TOULOUSE Jean Jaurès

PhD DEFENSE • SOFTWARE ENGINEERING

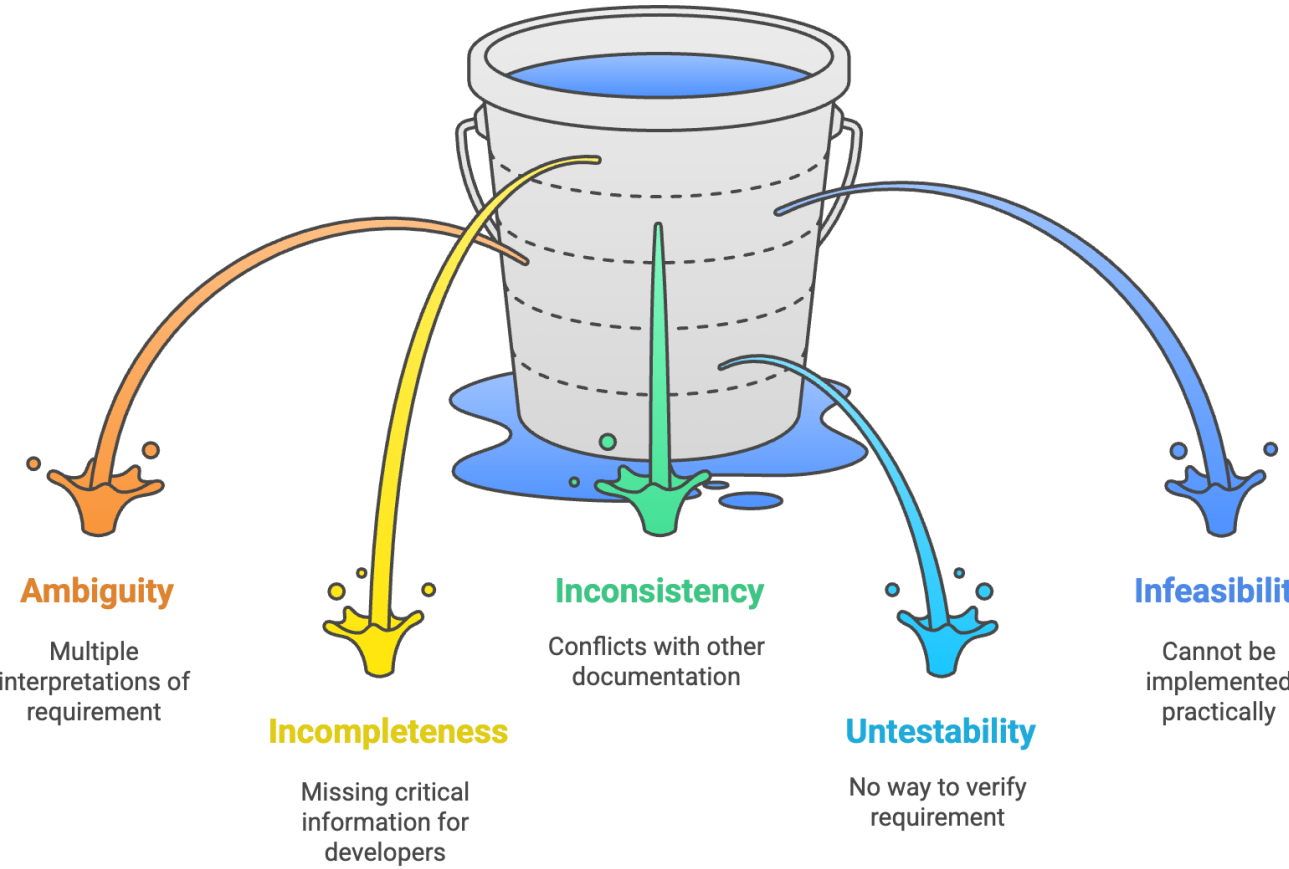
Requirements as Code

An object-oriented approach to security requirements

Ildar Nigmatullin

Supervisor: Pr. Sophie Ebersold
Co-supervisors: Dr. Nan Messe and Dr. Andrey Sadovykh
IRIT • Université Toulouse – Jean Jaurès

Hallmarks of Poor Requirements



Made with Napkin

https://www.jamasoftware.com/requirements-management-guide/requirements-management/guide-to-poor-requirements-identify-causes-repercussions-and-how-to-fix-them/

The thesis follows the manuscript's structure

1. Introduction and background
2. Related work
3. RQCODE
4. Two evaluations
5. Discussion & conclusion

01

Introduction & background

Why security requirements need a reliable bridge from written intent to evidence?

What counts as a security requirement?

Definition adopted in this thesis

A security requirement is a detailed requirement that implements an overriding security policy by constraining the system to preserve the confidentiality, integrity, and availability of its assets against identified threats.

Confidentiality

Information is not made available or disclosed to unauthorized individuals, entities, or processes.

Example: Client–server traffic must be encrypted with TLS 1.2 or higher.

Integrity

Accuracy and completeness of information.

Example: An update package must be rejected unless its digital signature verifies.

Availability

Accessible and usable on demand by an authorized entity.

Example: The login service must be available for 99.9% of each calendar month.

Definition: D. Firesmith, “Engineering Security Requirements”, Journal of Object Technology 2(1), 2003, extended with the CIA triad.
Confidentiality, integrity, availability: ISO/IEC 27000:2018, Information technology — Security techniques — Information security management systems — Overview and vocabulary.

Security requirements respond to measurable risk

Why security requirements?

Security requirements make expected protection explicit and checkable.

- **31% of breaches** — Started with vulnerability exploitation in Verizon's 2026 DBIR, making it the leading breach entry point.
- **>48,000 new CVEs** — Published in 2025, 22% more than in 2024, according to ENISA's 2026 Threat Landscape.

Sources: Verizon 2026 DBIR (2025 breach data); ENISA Threat Landscape 2026 (2025 CVE data).

Security requirements are written in one phase of SDLC and checked in another

Security Requirement Specification phase

Natural Language statements in security standards, policies, and guides.

Implementation phase

Code, configuration, scripts, and operational checks.

The link between the security requirements specification and its implementation is managed by people

Sources: Ruiz, Hu & Dalpiaz (2023), Requirements Engineering 28(4); Ramesh & Jarke (2001), IEEE Trans. Software Engineering 27(1).

Four motivation drivers of the thesis

MD1 **Precision of NL requirements**

NL security requirements are often imprecise, ambiguous, incomplete, or inconsistently stated. As a result, they cannot be automatically checked against the systems they describe.

MD2 **Traceability and evolution**

Security standards such as ISO/IEC 27001, OWASP, and DISA STIG are regularly revised. When traces between standards, security requirements, verification artifacts, and implementation artifacts are managed manually and outside version-controlled code, these artifacts drift apart.

MD3 **Reusability**

Security requirements are highly context-dependent. Generic templates must be adapted before they can be applied, while project-specific requirements are distributed across artifacts and projects, which makes their coordinated evolution difficult to manage.

MD4 **Usability**

Evidence cited in the thesis problem statement: P1 Zhao 2021; Guo 2025. P2 Mubarkoot 2023; Castellanos 2022. P3 Sönmez 2021.

Five contributions in the thesis

C1	**A Conceptual framework**	A framework for representing security requirements as executable OO artifacts, combining requirement specification and verification within a unified formalism.
C2	**An End-to-end formalization process**	A process for transforming NL security requirements into executable OO artifacts while preserving traceability between requirements, verification mechanisms, and implementation artifacts.
C3	**A Reusable requirement artefacts mechanism**	A mechanism for developing reusable security requirement artifacts through OO specialization, allowing existing requirements to be reused and adapted to different contexts while preserving their verification capabilities.
C4	**An Industrial evaluation**	An industrial evaluation of RQCODE conducted with a VeriDevOps project partner, demonstrating the application of the approach to real-world security requirements.
C5	**A Software practitioner study**	An empirical evaluation involving software practitioners to assess the applicability, usability, and perceived usefulness of RQCODE.

C1–C3 define the approach; C4–C5 provide initial evidence on feasibility and perceived usefulness.

02

Related work

Assess requirements methods and approaches to identify their limitations

How the seven assessment criteria are chosen

P1	Precision of NL requirements	→	NFR formalization	SRS V&V
P2	Traceability and evolution	→	SRS maintenance	Seamlessness
P3	Reusability	→	SRS reusability	
	Adoption constraint	→	Practitioner friendliness	Tool support

Retained or adapted from Bruel et al. (2021)

Defined for this thesis

Bruel et al. (2021), “The role of formalism in system requirements”, ACM Computing Surveys.

None of the 13 approaches meets all seven criteria

Problem tested	P2	P1	P2	P1	P3	Adoption constraint	
Approach	**Maintenance**	**V&V**	**Seamlessness**	**NFR formalization**	**Reusability**	**Practitioner friendliness**	**Tool support**
SOOR	Partially	Partially	**Yes**	Partially	Partially	Partially	**Yes**
SIRCOD	Partially	**Yes**	**Yes**	Partially	Partially	Partially	**Yes**
Security Use Case	No	Partially	No	No	Partially	**Yes**	No
SysML	Partially	Partially	No	Partially	Partially	Partially	**Yes**
TDD	Partially	Partially	Partially	Partially	No	**Yes**	**Yes**
BDD	Partially	Partially	Partially	No	No	Partially	**Yes**
SREP	No	Partially	No	Partially	**Yes**	Partially	Partially
OOAD	Partially	Partially	No	Partially	No	**Yes**	**Yes**
Secure Tropos	Partially	Partially	No	Partially	No	Partially	**Yes**
SQUARE	No	Partially	No	No	Partially	Partially	Partially
VICE	Partially	Partially	No	Partially	Partially	Partially	**Yes**
Struct. crypto spec.	Partially	Partially	No	**Yes**	No	Partially	**Yes**
Kant	Partially	Partially	No	Partially	Partially	Partially	**Yes**

Takeaway: Tool support is the only criterion most approaches meet (10 of 13 rated Yes). No approach is rated Yes on maintenance, and V&V, NFR formalization, and reusability have one each.

Ratings: the author's assessment of each approach's published description (thesis Table 3.1). Selection: 125 identified, 61 screened, 30 eligible, 13 assessed.

Existing approaches solve parts of the problem

Seamless approach

SOOR, SIRCOD

Provides: requirements as classes in the implementation language; the only approaches rated Yes on seamlessness.

Leaves open: SRS maintenance, NFR formalization, and SRS reusability only partial; Eiffel ecosystem.

TDD and BDD

TDD, BDD

Provides: executable checks with familiar tools; Yes on tool support, TDD also on practitioner friendliness.

Leaves open: the requirement is not a first-class artifact; both rated No on SRS reusability.

UML-based, OO-based, goal-oriented, Common Criteria, and multilateral approaches

Security Use Case, SysML, OOAD, Secure Tropos, SREP, SQUARE

Provides: structure and analysis of requirements; SREP rated Yes on SRS reusability.

Leaves open: all six rated No on seamlessness: the specification stays apart from verification.

Cryptographic protocol verification

VICE, Structured Cryptographic Protocol Specification, Kant

Provides: formal specification of protocol properties, with tool support.

Leaves open: all three rated No on seamlessness; scope limited to cryptographic protocols.

No approach is rated Yes on all seven criteria. Opportunity: an explicit requirement artifact, an object-oriented structure, and executable verification, in a familiar programming workflow.

Hypothesis: the OO paradigm suits security requirements

The OO paradigm is suitable for specifying and verifying security requirements, in a language that practitioners already use.

Why it is applicable?

From the related-work assessment

- Only the seamless approach (SOOR, SIRCOD) is rated Yes on seamlessness, and it is built on Eiffel.
- Practitioner friendliness is rated Yes for OOAD, TDD, and Security Use Case: notations that practitioners already know.
- No approach is rated Yes on SRS maintenance, and only SREP on SRS reusability.

What OO would contribute?

Design rationale, to be tested

- **Encapsulation:** the statement and its check in one class (P1, P2)
- **Specialization:** generic requirements adapted to each context (P3)
- **Interfaces:** one check() contract that tools can run (V&V)
- **Mainstream language:** no new notation to learn (adoption constraint)

Two questions guide the thesis

RQ1 • How?

How to formalize security requirements as executable Object-Oriented (OO) artifacts that integrate specification and verification within the software development process?

RQ2 • To what extent?

Does the approach improve practitioners' ability to formalize requirements in usability, verification support, and effort?

03

RQCODE

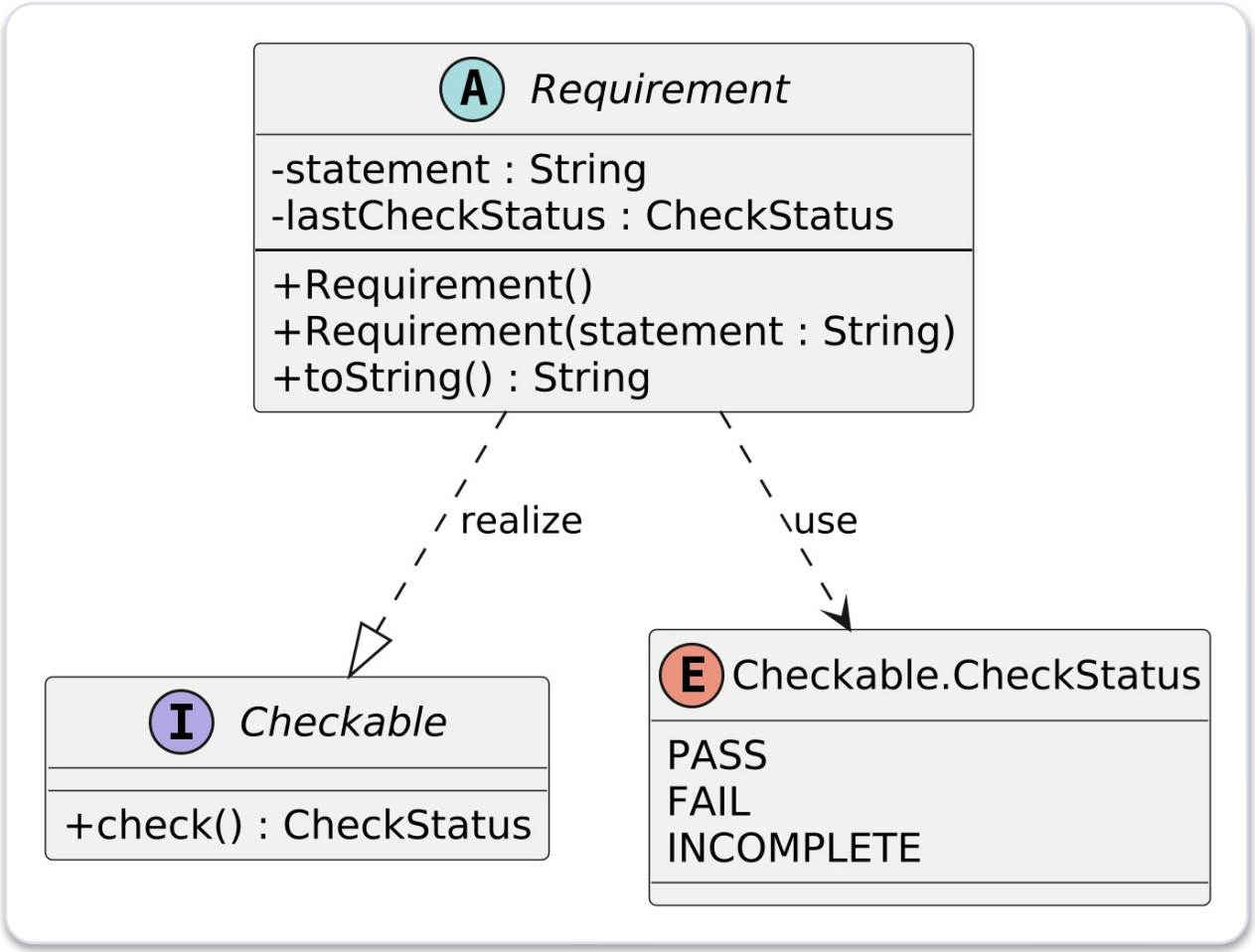
Trace the security requirement from its statement to a checkable object-oriented artifact

RQCODE definition

RQCODE is an approach supported by an **object-oriented structure** and implemented through a **Java framework**.

Together, they combine a requirement's specification and its verification in one software artifact.

Each requirement carries its statement and its check



Source: thesis, Figure 4.1. Concrete requirements extend Requirement and implement check().

A **Requirement** *abstract class*

Holds the statement in NL text and the result of the last check (lastCheckStatus).

I **Checkable** *interface*

Declares check(), which inspects the system under verification and returns a CheckStatus.

E **CheckStatus** *enum*

PASS: the property holds. FAIL: it does not. INCOMPLETE: no verdict is possible, for example when the state cannot be observed.

Why lightweight formalization is needed

Natural language

+ Readable and writable by every stakeholder
− Imprecise and ambiguous, so it cannot be checked automatically (P1)
− Traces and reuse stay manual (P2, P3)

RQCODE

Lightweight formalization

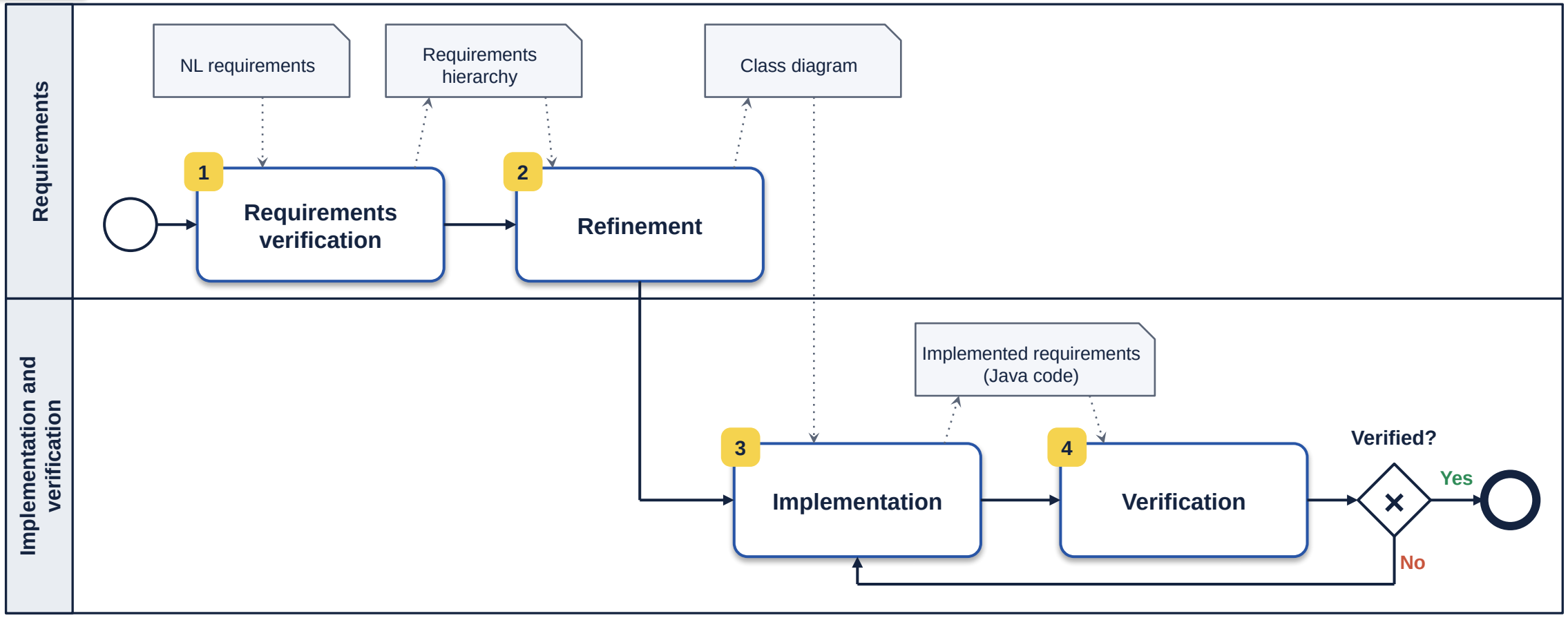
+ Precise enough for automated checking of selected properties
+ Written in a language and toolchain developers already use
− Partial: selected properties only, no proof of correctness

Full formal methods

+ Complete, mathematically precise specification
− Learning curve and limited tool integration have hindered industrial adoption

Basis: the thesis problem statement (P1–P3 and the adoption constraint).

Formalization is a four-operation, iterative process



- Requirements verification examines the NL requirements; Verification examines the system under verification.
- The process is selective: when the input is already atomic and precise, Implementation and Verification alone may suffice.
- BPMN only describes the process; it is not part of RQCODE.

Example: a security requirement becomes an executable class

Starting point: NL requirement

“Lock the account after five consecutive failed logins.”

Implemented requirement (simplified Java sketch)

```
public class AccountLockout extends Requirement {

  public AccountLockout() {
    super("Lock the account after five "
        + "consecutive failed logins.");
  }

  @Override
  public CheckStatus check() {
    if (!stateObservable())
      return CheckStatus.INCOMPLETE;
    return isLocked() ? CheckStatus.PASS
                      : CheckStatus.FAIL;
  }
}
```

check() after the fifth failed login

PASS
The account is locked.

FAIL
The account is still unlocked.

INCOMPLETE
The counter or lock state cannot be observed.

The statement and its check live in one class; the verdict is only as trustworthy as the observation behind it.

Simplified sketch for illustration; helper names are illustrative. The class extends Requirement from the metamodel.

The Java framework runs in the ordinary toolchain

The framework implements the metamodel. Each requirement is a Java class built on its core types.

CORE TYPES

Requirement	Checkable	CheckStatus	Enforceable	EnforcementStatus
statement, last result	check()	PASS, FAIL, INCOMPLETE	enforce(), optional	SUCCESS, FAILURE, INCOMPLETE

RUNS IN THE ORDINARY TOOLCHAIN

Java
Requirement classes and checks

Git
Version history of each requirement class

JUnit
Checks can run as tests

Command line
Run a check on a target

CI/CD
Planned: run checks in the delivery pipeline

Open source: github.com/VeriDevOps/RQCODE

Demonstrated in this work: Java, Git, JUnit, and command-line execution. CI/CD integration is future work and has not been evaluated.

From SOOR to RQCODE: security requirements in Java

SOOR is the closest antecedent: a requirement is a class in the implementation language, with its check next to its specification. RQCODE applies this idea to security requirements in Java.

	SOOR (Naumchev et al., 2019)	RQCODE
REQUIREMENTS	Functional requirements	**Security requirements**
HOST LANGUAGE	Eiffel	**Java**
EXECUTABLE CHECK	Contracts built into the language	**Explicit check() method that any Java toolchain can invoke**

What differs: the requirement domain, the host language, and the form of the check.

Why lightweight formalization is needed

Natural language

+ Readable and writable by every stakeholder
− Imprecise and ambiguous, so it cannot be checked automatically (P1)
− Traces and reuse stay manual (P2, P3)

RQCODE

Lightweight formalization

+ Precise enough for automated checking of selected properties
+ Written in a language and toolchain developers already use
− Partial: selected properties only, no proof of correctness

Full formal methods

+ Complete, mathematically precise specification
− Learning curve and limited tool integration have hindered industrial adoption

Basis: the thesis problem statement (P1–P3 and the adoption constraint).

Lightweight formalization: four defining properties

Lightweight formalization is the specification of a requirement in a form precise enough for automated checking, while retaining four properties that distinguish it from full formal methods.

1 Selective scope

Chosen properties of a requirement are formalized, not a complete model of the system.

In RQCODE: one class checks the property its statement expresses.

2 Executable checking in place of proof

Conformance is decided by executing code, not by discharging proof obligations.

In RQCODE: check() returns PASS, FAIL, or INCOMPLETE.

3 Programming-language notation

The requirement is written in a general-purpose language already used for development.

In RQCODE: requirements are Java classes.

4 Low adoption cost

The representation fits ordinary environments and toolchains, with no dedicated formal-methods infrastructure.

In RQCODE: classes live in the codebase and run with standard Java tools.

Partially formal: more precise than NL, less complete than a mathematical specification.

Related to seamlessness (continuity of notation across the lifecycle) but not the same property: it concerns how completely a requirement is formalized. The thesis defines the term because “lightweight” has no shared definition (Zamansky et al., 2018).

How RQCODE responds to the problems

P1	**Precision of NL requirements**	→	Each requirement is an executable artifact: its statement is paired with a check() that returns PASS, FAIL, or INCOMPLETE.	C1
P2	**Traceability and evolution**	→	Statement, check, and implementation sit in one class under version control, so traces can be kept with the code as it evolves.	C2
P3	**Reusability**	→	Specialization lets related requirements share checking logic and adapt it to different contexts.	C3
	Adoption constraint	→	Lightweight formalization: Java notation, executable checks, and the standard development toolchain.	C1–C3

04

Evaluation

How RQCODE can be applicable for industrial domain and for software practitioners

Two studies answer different parts of the evaluation

Industrial study

Windows 10 Security Technical Implementation Guide (STIG)
One industrial partner (Fagor)
2+ years project
Two interview rounds

Practitioner study

22 MSc software engineering students
1.5-hour workshop + assignment
Survey and open responses

The industrial study applied RQCODE to one real security guide

Study context

- **Partner: Fagor, within the VeriDevOps project**
- Corpus: 256 Windows 10 STIG findings
- Period: almost two years, iterative implementation and feedback
- Evidence: implementation results, demonstration, two interviews

Question: how much of this corpus can be represented as executable classes, and what structure supports reuse?

One finding becomes metadata, a check, and a remediation path

STIG finding

Overview: identifier and severity
Check Text: how to observe compliance
Fix Text: how to remediate

RQCODE class

Metadata map
check() delegates verification
enforce() delegates remediation

Observed output

PASS
FAIL
INCOMPLETE

RQCODE formalized 165 of 256 findings in the study corpus

64.4%



Formalized 165 Not formalized: 91

Audit Policy

31 findings

Registry Edit

106 findings

User Rights

28 findings

These figures describe this Windows 10 STIG snapshot only.

Reuse and traceability are visible in the implementation structure

AuditPolStig

Shared verification and enforcement logic for a category

V_220778.java → STIG ID V-220778

Finding parameters + metadata
Version history in Git
Re-run check() after a change

Mechanism shown: reuse and trace links. Outcome not measured: maintenance effort saved.

The design supports two different kinds of change

One finding changes

Edit the finding-specific value and metadata.
Commit links the changed class to its finding ID.

Shared category logic changes

Edit the category class once.
Rerun checks for all affected findings.

A plausible maintenance path, illustrated by the architecture—not an empirically measured savings claim.

The partner feedback is positive but comes from one collaboration

Reported strengths

Reuse, extension, traceability, validation, and maintenance were rated positively during formalization.

Reported friction

Java/programming familiarity and setup effort remained concerns.

Useful case evidence and practitioner insight, not a general usability estimate. Need a broader evaluation

The practitioner study tested initial learning and perceptions

22 participants

MSc software engineering students

1.5 h workshop

security concepts + RQCODE demo

1 home assignment

Password lockout requirement

14 open answers

response counts vary by question

Participants most often agreed on verification, traceability, and reuse



Perceived agreement; no control group and no objective comparison of effort or defect detection.

The open responses show both value and adoption friction

What helped

Hands-on implementation; worked examples, especially the lockout case; a self-contained reference.

What got in the way

Tutorial setup/build problems; unfamiliarity with Java; desire for workflow integration examples.

Design implication: onboarding, tested templates, and workflow integration deserve further study.

05

Discussion

Interpret feasibility and perceived usefulness, then make the limits of the evidence explicit.



The evidence answers RQ1 more strongly than it answers impact

RQ1 • Feasibility

A defined representation and process;
Applied to 165 findings in one Windows 10 STIG corpus.

RQ2 • Initial evidence

Reuse and traceability mechanisms demonstrated;
Participants and partner reported positive perceptions.

Not established: lower maintenance cost, fewer defects, or superiority to alternative methods.

The main limitations define where the conclusions stop

Automation scope

91/256 findings were not formalized;
Human and organizational context remains.

Initial cost

Hierarchy design and Java/OOP learning overhead were not quantified.

Participants

MSc students provide early learnability evidence, not expert security-engineering validation.

External validity

One partner, OS, standard, and small cohort; no controlled comparison.

The defensible conclusion is feasibility plus promising perceived usefulness—not proven superiority.

RQCODE fits requirements with observable properties and actions

Stronger fit

A requirement maps to state or behavior that software can observe, with a repeatable check and clear evidence.

Conditional fit

Human judgment, organizational procedure, external context, or incomplete observability needs complementary review.

Positioning RQCODE against the 13 approaches

Approach	Maintenance	V&V	Seamlessness	NFR formalization	Reusability	Practitioner friendliness	Tool support
SOOR	Partially	Partially	**Yes**	Partially	Partially	Partially	**Yes**
SIRCOD	Partially	**Yes**	**Yes**	Partially	Partially	Partially	**Yes**
Security Use Case	No	Partially	No	No	Partially	**Yes**	No
SysML	Partially	Partially	No	Partially	Partially	Partially	**Yes**
TDD	Partially	Partially	Partially	Partially	No	**Yes**	**Yes**
BDD	Partially	Partially	Partially	No	No	Partially	**Yes**
SREP	No	Partially	No	Partially	**Yes**	Partially	Partially
OOAD	Partially	Partially	No	Partially	No	**Yes**	**Yes**
Secure Tropos	Partially	Partially	No	Partially	No	Partially	**Yes**
SQUARE	No	Partially	No	No	Partially	Partially	Partially
VICE	Partially	Partially	No	Partially	Partially	Partially	**Yes**
Struct. crypto spec.	Partially	Partially	No	**Yes**	No	Partially	**Yes**
Kant	Partially	Partially	No	Partially	Partially	Partially	**Yes**
RQCODE	**Yes**	**Yes**	**Yes**	**Partially**	**Yes**	**Yes**	**Yes**

Takeaway: RQCODE is rated Yes on six criteria and Partially on NFR formalization; no other approach is rated Yes on more than three.

vs. SOOR, SIRCOD: same seamless principle, in Java rather than Eiffel.

vs. TDD, BDD: the requirement stays a separate artifact that is executed.

NFR partial: only requirements reducible to executable predicates.

Ratings: the author's assessment (thesis Tables 3.1 and 6.1). RQCODE is rated by its author on the same scale, so the table is a positioning, not an independent comparison.

06

Conclusion

Recap the model, process, reusable structure, industrial application, and practitioner study.

Contributions and how they were addressed

	CONTRIBUTION	HOW IT WAS ADDRESSED
C1	**Conceptual foundation**	Metamodel and Java framework: each requirement holds its NL statement and an executable check() with three outcomes, PASS, FAIL, and INCOMPLETE.
C2	**End-to-end formalization process and traceability**	Four-operation process (requirements verification, refinement, implementation, verification), applied to Windows 10 STIG findings; each finding ID traces to a class and its Git history.
C3	**Reusable requirement artifacts**	OO specialization: category classes hold the shared checking logic; in the industrial study, one AuditPolStig class serves 31 Audit Policy findings.
C4	**Industrial study**	With Fagor in VeriDevOps: 165 of 256 Windows 10 STIG findings formalized as executable classes; two interview rounds gave positive feedback, with Java and setup friction.
C5	**Software practitioner study**	22 MSc software engineering students, after a workshop and an assignment: 91% agreed on traceability and on reuse, 86% on easier verification, 62% on check() reliability.

Together: feasibility is shown and perceived usefulness is promising; lower cost or superiority is not established.

How the research questions were answered

RQ1 How?

How can security requirements be formalized as executable object-oriented artifacts?

ANSWER

As Java classes that keep the NL statement and an executable check(), produced by a four-operation process and organized by OO specialization.

EVIDENCE

- Metamodel, process, and Java framework defined and implemented (C1–C3)
- 165 of 256 Windows 10 STIG findings formalized (64.4%)
- Applied with one industrial partner over almost two years
- Limit: 91 findings not formalized

Answered for one industrial corpus

RQ2 To what extent?

Does the approach improve practitioners' ability to formalize requirements in usability, verification support, and effort?

ANSWER

On initial evidence only: perceptions of verification support are positive, usability shows friction, and effort was not compared.

EVIDENCE

- Verification support: 86% agreed that verifying is easier; 62% that check() is reliable
- Usability: worked examples helped; Java familiarity and setup got in the way
- Industrial partner: positive feedback within a project

Answered in part: initial, perception-based evidence

Overall: the approach is feasible; its impact on practice remains to be measured with professionals and controlled tasks.

Short-term and long-term perspectives

Short term

Engineering work guided by the two studies

Graphical user interface — L2

Non-technical practitioners view and change requirements, with RQCODE as the backend, without editing Java code.

Language-neutral API — L2

External tools use RQCODE without implementing it in Java.

Larger template library — L1 L2

Parent classes for more categories of security requirements, so a new requirement reuses a template.

Long term

Research directions that extend the scope of RQCODE

Hybrid verification — L1

An executable check for the technical part, a structured inspection for the rest; the specification–verification link remains.

LLM-assisted formalization — L2

The model suggests the parent class and parameters; compilation and execution checks decide acceptance.

CI/CD and DevSecOps integration

Checks run in builds and deployments, so violations are found continuously, not only at compliance assessments.

Tags: the limitation of the approach each direction addresses (L1 scope of automation, L2 initial setup).

Limitations of the evaluation (L3, L4): they call for replication with professional security and requirements engineers across several organizations and standards, not for a change to the approach.

Thank you

Questions and discussion

Ildar Nigmatullin • IRIT • Université Toulouse – Jean Jaurès